CLASSIFICATIONS PROBABILISTES

Le but de ce projet est d'étudier les méthodes de classifications auxquelles les statistiques donnent accès naturellement. Il s'agira donc d'étudier les outils de classifications probabilistes. C'est bien évidemment une petite partie de l'ensemble des méthodes de classification existant.

In [1]:
```
%load_ext autoreload
%autoreload 2

# utils.py contient des fonctions et des classes d'aide au tme8. IL NE FAUT RIEN CHANGER DANS CE FICHIER
# liste des classes et fonctions de ce fichier :
#  - getNthDict (remplit un dictionnaire avec la ligne n d'un dataframe)
#  - viewData (visualisation d'un dataframe)
#  - discretizeData (discrétisation automatique)
#  - AbstractClassifier (classe abstraite pour tous les classifieurs)
#  - drawGraphHorizontal (dessin d'un graphe orienté horizontal)
#  - drawGraph (dessin d'un graphe orienté vertical)
import utils

import tme8
```

Base utilisée : heart disease (Cleveland database)

```
    This database contains 76 attributes, but all published experiments refer to using a subset of 14 of them. In particular, the Cleveland database is the only
one that has been used by ML researchers to this date.
    The `target` field refers to the presence of heart disease in the patient. It is integer valued from 0 (no presence) to 4. Experiments with the Cleveland
database have concentrated on simply attempting to distinguish presence (values 1) from absence (value 0).
```

champs	definition
age	age in years
sex	(1 = male; 0 = female)
cp	chest pain type
trestbps	resting blood pressure (in mm Hg on admission to the hospital)
chol	serum cholestoral in mg/dl
fbs	(fasting blood sugar > 120 mg/dl) (1 = true; 0 = false)
restecg	resting electrocardiographic results
thalach	maximum heart rate achieved
exang	exercise induced angina (1 = yes; 0 = no)
oldpeak	ST depression induced by exercise relative to rest
slope	the slope of the peak exercise ST segment
ca	number of major vessels (0-3) colored by flourosopy
thal	3 = normal; 6 = fixed defect; 7 = reversable defect
target	1 or 0

Notre but est donc de proposer des classifieurs qui tentent de prédire la valeur de `target` à partir des autres champs en utilisant des arguments probabilistes.

0- Simplification de la base

prélude au projet : pas de travail à faire dans cette partie

Nous allons prétraiter la base afin de la rendre utilisable pour notre tme8. Le travail se fait pour vous. Rien à faire, juste à regarder pour comprendre ce qu'on y fait.

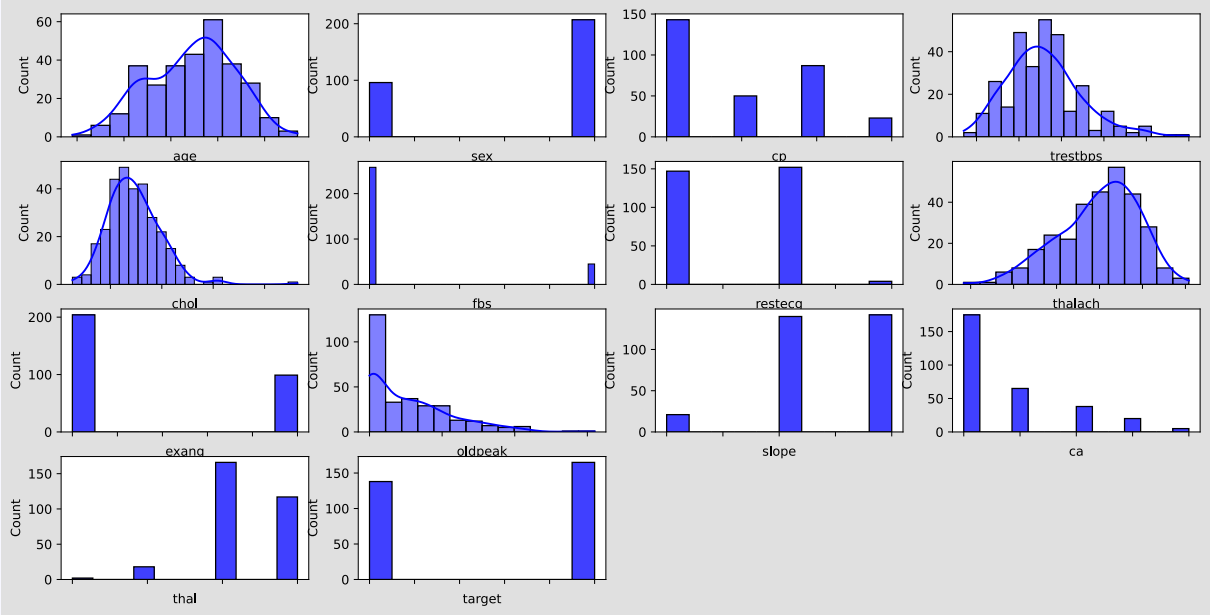
In [2]:
```
import pandas as pd # package for high-performance, easy-to-use data structures and data analysis
import numpy as np # fundamental package for scientific computing with Python
```

In [3]:
```
data=pd.read_csv("data/heart.csv")
data.head()
```

Out[3]:

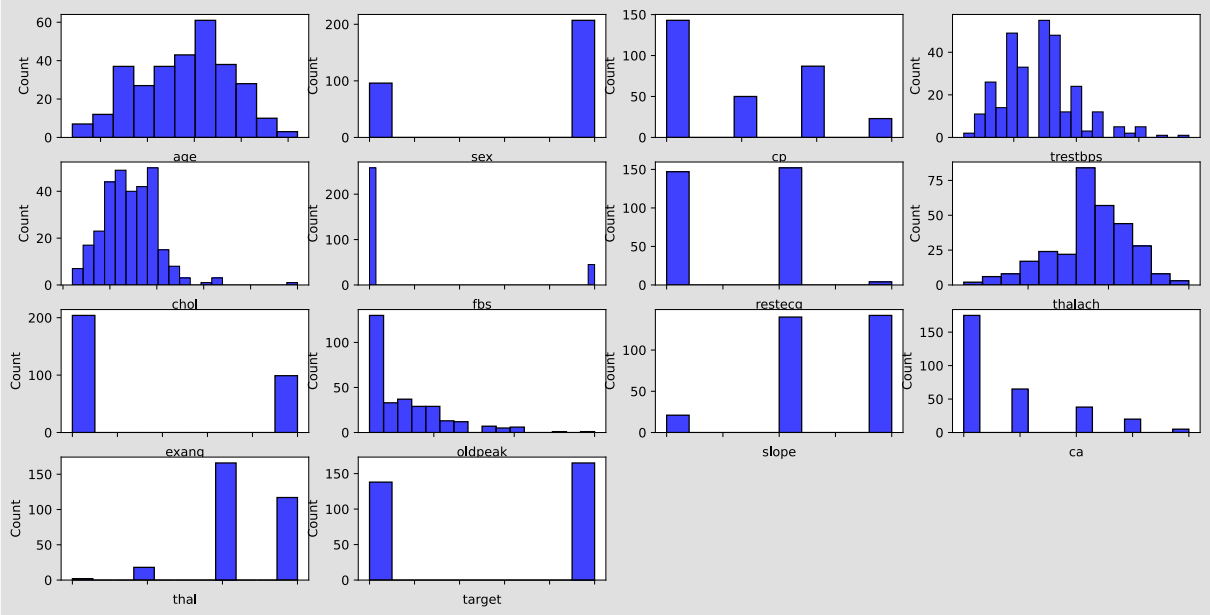
	age	sex	cp	trestbps	chol	fbs	restecg	thalach	exang	oldpeak	slope	ca	thal	target
0	63	1	3	145	233	1	0	150	0	2.3	0	0	1	1
1	37	1	2	130	250	0	1	187	0	3.5	0	0	2	1
2	41	0	1	130	204	0	0	172	0	1.4	2	0	2	1
3	56	1	1	120	236	0	1	178	0	0.8	2	0	2	1
4	57	0	0	120	354	0	1	163	1	0.6	2	0	2	1

In [4]:
```
utils.viewData(data)
```



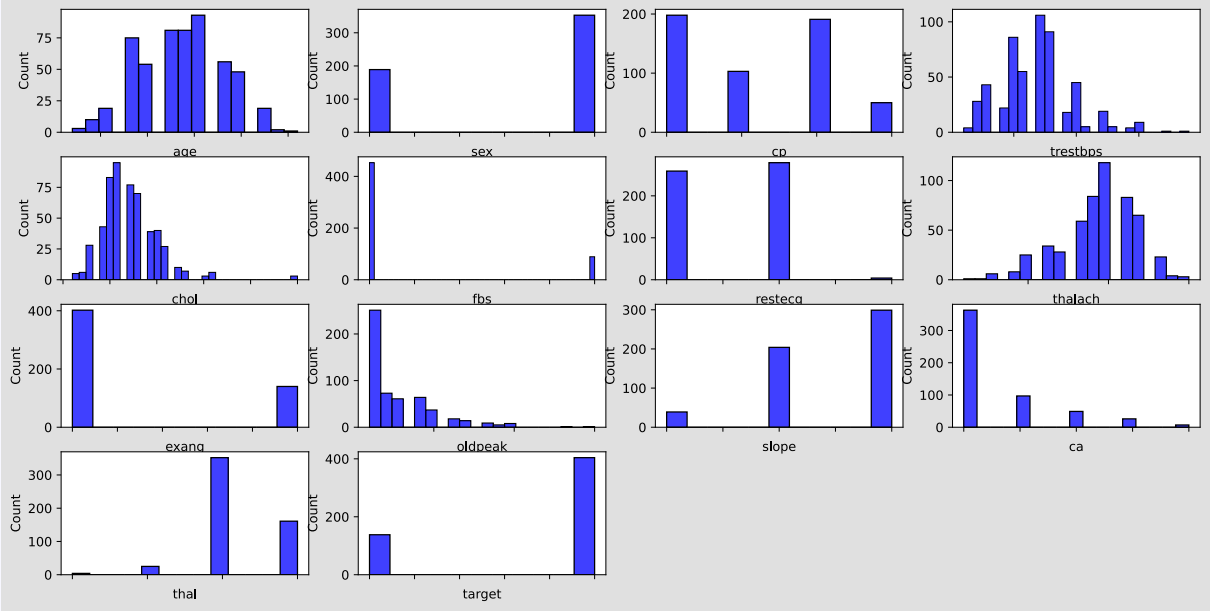
Certaines variabless comme `age`, `thalach`, etc. possèdent un grand nombre de modalités, rendant difficile le traitement. Nous simplifions donc la base en discrétisant au mieux toutes les variables qui ont plus de 5 valeurs.

In [5]:
```
discretise=utils.discretizeData(data)
utils.viewData(discretise,kde=False)
```



Nous utilisons maintenant 2 fichiers csv **préparés à l'avance pour ce projet** à partir de cette base afin de rendre les résultats plus intéressants (en particulier, les 2 classes sont un peu plus déséquilibrées).

In [6]:
```
train=pd.read_csv("data/train.csv")
test=pd.read_csv("data/test.csv")
utils.viewData(train,kde=False)
```



À partir de maintenant, nous utilisons le dataframe `train` qui contient les données pour l'apprentissage et `test` qui contient les données pour la validation.

1- Classification a priori

Question 1.1 : calcul de la probabilité a priori

Dans une fonction `getPrior`, calculer la probabilité a priori de la classe 1 ainsi que l'intervalle de confiance à 95% pour l'estimation de cette probabilité.

In [7]:
```
# cette fonction doit donc rendre un dictionnaire contenant 3 clés 'estimation', 'min5pourcent', 'max5pourcent'
# (L'ORDRE DES CLES N'EST PAS IMPORTANTE DANS UN DICTIONNAIRE)
tme8.getPrior(train)
```

Out[7]:
```
{'estimation': 0.7453874538745388,
 'min5pourcent': 0.7087109975695709,
 'max5pourcent': 0.7820639101795066}
```

In [8]:
```
tme8.getPrior(test)
```

Out[8]:
```
{'estimation': 0.69,
 'min5pourcent': 0.62590170673099,
 'max5pourcent': 0.7540982932690099}
```

Question 1.2 : programmation orientée objet dans la hiérarchie des `Classifier`

On propose de représenter les classifieurs en python par des classes d'une hiérarchie. Un classifieur répond à une question principale : étant donné un vecteur d'attributs, quelle est la classe proposée ? Nous proposons donc une classe de base qu'il s'agira d'**améliorer et de spécialiser en la sous-classant** : `AbstractClassifier` dans le fichier `utils.py`

Telle qu'elle est définie dans `utils.py`, la classe `AbstractClassifier` ressemble à une *interface* (en Java) : elle décrit sans implémenter les deux méthodes que doivent implémenter les `Classsifier` que vous allez écrire.

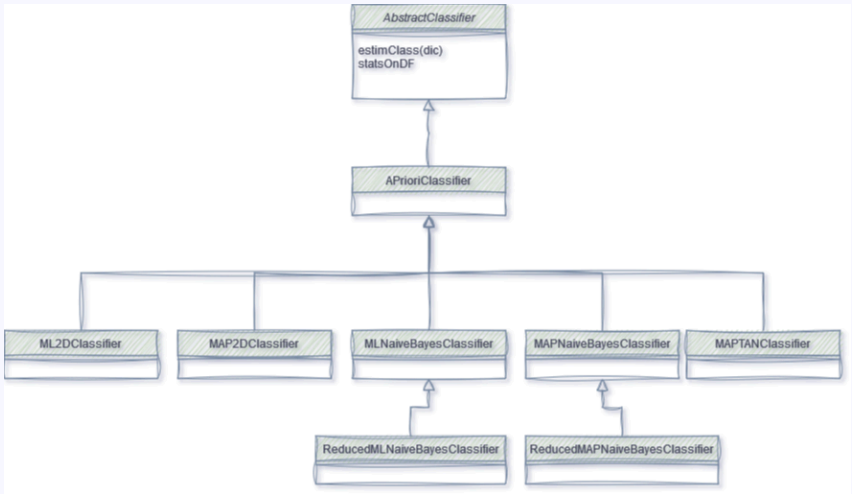
- `estimClass` qui à partir d'un dictionnaire tel que :

  `{'age': 9, 'sex': 1, 'cp': 3, 'trestbps': 9, 'chol': 6, 'fbs': 1, 'restecg': 0, 'thalach': 9, 'exang': 0, 'oldpeak': 6, 'slope': 0, 'ca': 0, 'thal': 1, 'target': 1}`

  sera capable de prédire si le patient semble malade ou non en retournant soit `1` soit `0`. (que l'on connait grâce à `target` qu'il ne faut pas utiliser dans le classifieur bien sûr ! 😄)

- `statsOnDF` qui fournit des statistiques de qualité du `Classifier` en le confrontant à une base de donnée (un `pandas.DataFrame`).

Voici un schéma des classes que vous allez pouvoir créer dans ce projet (en n'affichant que le nom des classes et non le nom des méthodes à implémenter dans chacune):



Question 1.2.a

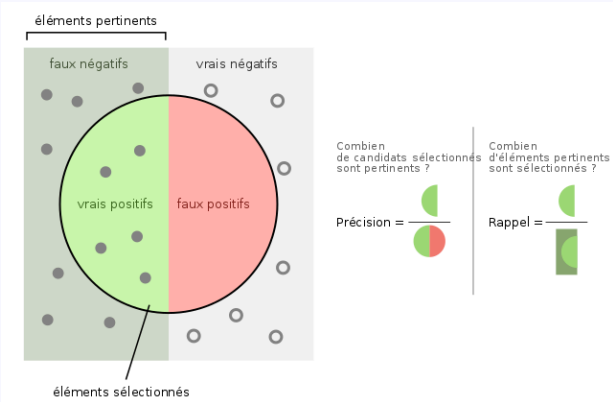
Ecrire dans `tme8.py` un classifieur `APrioriClassifier` (enfant de `AbstractClassifier`) qui utilise le résultat de la question 1 pour estimer très simplement la classe de chaque individu par la classe majoritaire.

In [9]:
```
cl=tme8.APrioriClassifier(train)
clpredite=cl.estimClass({}) # n'importe quoi donne la même classe pour un classifieur a priori
# la valeur prédite n'est pas affichée sciemment (pour vous laisser chercher) mais doit valoir l'entier 0 ou 1
```

Question 1.2.b : évaluation de classifieurs

Implémenter également la méthode `statsOnDF` qui rendra les valeurs suivantes :

- VP : `vrai positif` . Le nombre d'individus avec target=1 et classe prévue=1
- VN : `vrai négatif` . Le nombre d'individus avec target=0 et classe prévue=0
- FP : `faux positif` . Le nombre d'individus avec target=0 et classe prévue=1
- FN : `faux négatif` . Le nombre d'individus avec target=1 et classe prévue=0
- précision
- rappel



- Petite aide : comment itérer sur un dataframe

```
for t in train.itertuples():
    dic=t._asdict()
    print(f"ca={dic['ca']} oldpeak={dic['oldpeak']} target={dic['target']}")
```

- Par ailleurs, dans `utils.py` , il y a une fonction `getNthDict(df,n)` qui rend le dictionnaire des attributs de la $n$ième ligne dans `df` .

```
>>> getNthDict(train,0)
{'age': 9, 'sex': 1, 'cp': 3, 'trestbps': 9, 'chol': 6, 'fbs': 1, 'restecg': 0, 'thalach': 9, 'exang': 0, 'oldpeak': 6, 'slope': 0, 'ca': 0, 'thal': 1,
'target': 1}
```

In [10]:
```
cl=tme8.APrioriClassifier(train)
print("test en apprentissage : {}".format(cl.statsOnDF(train)))
print("test en validation: {}".format(cl.statsOnDF(test)))
```

```
test en apprentissage : {'VP': 404, 'VN': 0, 'FP': 138, 'FN': 0, 'Précision': 0.7453874538745388, 'Rappel': 1.0}
test en validation: {'VP': 138, 'VN': 0, 'FP': 62, 'FN': 0, 'Précision': 0.69, 'Rappel': 1.0}
```

⚠ Si la méthode `statsOnDF` est correctement écrite, elle sera la même pour tous les prochains classifieurs. **Afin de ne pas avoir à réécrire cette méthode `statsOnDF` qui ne devrait pas changer, on fera hériter tous les classifieurs de `AprioriClassifier` plutôt que de `AbstractClassfier`** (voir le schéma en début de section 1.2).

2- classification probabiliste à 2 dimensions

La classification a priori ne donne pas d'excellents résultats puiqu'elle se contente de la règle majoritaire. On se propose donc maintenant essayer d'enrichir notre processus de décision en tenant compte d'une caractéristique de la base de données.

Question 2.1 : probabilités conditionelles

Question 2.1.a

Écrire une fonction `P2D_l(df,attr)` qui calcule dans le dataframe `df` la distribution de probabilité $P(attr|target)$ sous la forme d'un dictionnaire asssociant à la valeur t un dictionnaire associant à la valeur a la probabilité $P(attr=a|target=t)$.

In [11]:
```
p_thal_given_target=tme8.P2D_l(train,'thal') # calcul P(thal=a|target=t)

print(p_thal_given_target)
print()
print(f"Dans la base train, la probabilité que thal=3 sachant que target=1 est {p_thal_given_target[1][3]}")
```

```
{1: {1: 0.03217821782178218, 2: 0.7821782178217822, 3: 0.1782178217821782, 0: 0.007425742574257425}, 0: {1: 0.08695652173913043, 2: 0.2608695652173913, 3: 0.644927536231884, 0: 0.00724637
6811594203}}

Dans la base train, la probabilité que thal=3 sachant que target=1 est 0.1782178217821782
```

Question 2.1.b

Écrire une fonction `P2D_p(df,attr)` qui calcule dans le dataframe la probabilité $P(target|attr)$ sous la forme d'un dictionnaire associant à la valeur a un dictionnaire asssociant à la valeur t la probabilité $P(target=t|attr=a)$.

In [12]:
```
p_target_given_thal=tme8.P2D_p(train,'thal')

print(p_target_given_thal)
print()
print(f"Dans la base train, la probabilité que target=1 sachant que thal=3 est {p_target_given_thal[3][1]}")
```

```
{1: {1: 0.52, 0: 0.48}, 2: {1: 0.8977272727272727, 0: 0.10227272727272728}, 3: {1: 0.4472049689440994, 0: 0.5527950310559007}, 0: {1: 0.75, 0: 0.25}}

Dans la base train, la probabilité que target=1 sachant que thal=3 est 0.4472049689440994
```

Question 2.2 : classifieurs 2D par maximum de vraisemblance

Supposons qu'un individu ait la valeur a pour $attr$, un classifieur du type $P2D_l$ pourrrait donc utiliser $P(attr=a|target=t)$ et sélectionner comme estimation de la classe de l'individu la valeur $t=0$ ou $t=1$ maximisant cette probabilité. $P(attr=a|target)$ est la vraisemblance d'observer $attr=a$ quand $target=0$ ou $target=1$. Un tel classifieur utilise donc le principe du **maximum de vraisemblance** (ML=Max Likelihood).

- Pour construire un tel classifieur (dans la méthode `__init__`), il faut initialiser l'attribut utilisé puis construire la table `P2Dl` .
- La fonction `estimClass` rendra la position du maximum trouvé dans cette table.

> Supposons un individu dont $thal=3$, alors dans la table P2Dl, on trouve 0.178 pour $target=1$ et 0.644 pour $target=0$, la bonne classe d'après le critère du ML est donc 0

Écrire une classe `ML2DClassifier` qui utilise une telle procédure de maximum de vraisemblance pour estimer la classe d'un individu. Afin de ne pas avoir à réécrire la méthode `statsOnDF` qui ne devrait pas changer, `ML2DClassifier` aura pour parent la classe `APrioriClassifier`.

PS- penser bien à calculer une seule fois la table `P2Dl` dans le constructeur de la classe afin de ne pas itérer sur toute la base à chaque fois que vous appelez la méthode `estimClass`.

PS2- Dans les cas de la stricte égalité des 2 probabilités, on choisira la classe 0.

In [13]:
```
cl=tme8.ML2DClassifier(train,"thal") # cette ligne appelle tme8.P2Dl(train,"thal")
for i in [0,1,2]:
    print("Estimation de la classe de l'individu {} par ML2DClassifier : {}".format(i,cl.estimClass(utils.getNthDict(train,i))))
```

```
Estimation de la classe de l'individu 0 par ML2DClassifier : 0
Estimation de la classe de l'individu 1 par ML2DClassifier : 1
Estimation de la classe de l'individu 2 par ML2DClassifier : 1
```

In [14]:
```
print("test en apprentissage : {}".format(cl.statsOnDF(train)))
print("test en validation: {}".format(cl.statsOnDF(test)))
```

```
test en apprentissage : {'VP': 319, 'VN': 101, 'FP': 37, 'FN': 85, 'Précision': 0.8960674157303371, 'Rappel': 0.7896039603960396}
test en validation: {'VP': 113, 'VN': 48, 'FP': 14, 'FN': 25, 'Précision': 0.889763779527559, 'Rappel': 0.8188405797101449}
```

Question 2.3 : classifieurs 2D par maximum a posteriori

Supposons qu'un individu ait la valeur a pour l'$attr$, un classifieur du type $P2D_p$ pourrrait donc utiliser $P(target = t|attr = a)$ et sélectionner comme estimation de la classe de l'individu la valeur $t = 0$ ou $t = 1$ maximisant cette probabilité. $P(target|attr = a)$ est la distribution a posteriori de $target$ après avoir observé $attr = a$. Un tel classifieur utilise donc le principe du **maximum a posteriori** (MAP).

Pour construire un tel classifieur, il faut initialiser l'attribut utilisé puis construire la table `P2Dp`. La fonction `estimClass` rendra la position du maximum trouvé dans cette table.

> Supposons un individu dont $thal = 3$, alors dans la table P2Dp, on trouve 0.447 pour $target = 1$ et 0.552 pour $target = 0$, la bonne classe d'après le critère du MAP est donc 0

Écrire une classe `MAP2DClassifier` qui utilise une telle procédure de maximum a posteriori pour estimer la classe d'un individu. Afin de ne pas avoir à réécrire la méthode `statsOnDF` qui ne devrait pas changer, `MAP2DClassifier` héritera de `AprioriClassifier`.

PS- penser bien à calculer une seule fois la table `P2Dp` dans le constructeur afin de ne pas itérer sur toute la base à chaque fois que vous appelez la méthode `estimClass`.

PS2- Dans les cas d'égalité des 2 probabilités, on choisira la classe 0.

In [15]:
```
cl=tme8.MAP2DClassifier(train,"thal") # cette ligne appelle tme8.P2Dp(train,"thal")
for i in [0,1,2]:
    print("Estimation de la classe de l'individu {} par MAP2DClasssifer) : {}".format(i,cl.estimClass(utils.getNthDict(train,i))))
```

```
Estimation de la classe de l'individu 0 par MAP2DClasssifer) : 1
Estimation de la classe de l'individu 1 par MAP2DClasssifer) : 1
Estimation de la classe de l'individu 2 par MAP2DClasssifer) : 1
```

In [16]:
```
print("test en apprentissage : {}".format(cl.statsOnDF(train)))
print("test en validation: {}".format(cl.statsOnDF(test)))
```

```
test en apprentissage : {'VP': 332, 'VN': 89, 'FP': 49, 'FN': 72, 'Précision': 0.8713910761154856, 'Rappel': 0.8217821782178217}
test en validation: {'VP': 114, 'VN': 43, 'FP': 19, 'FN': 24, 'Précision': 0.8571428571428571, 'Rappel': 0.8260869565217391}
```

Question 2.4 : comparaison

Quelle classifieur préférez-vous en théorie entre `APrioriClassifier`, `ML2DClassifier` et `MAP2DClassifier` ? Quels résultats vous semble-les plus intéressants ?

Cette question ouverte doit donc être répondue dans `tme8.py` sous la forme :

```
#####
# Question 2.4 : comparaison
#####
# Nous préférons ... parce que ...
# et aussi parce que ...
#####
```

In [ ]: